PROVISIONAL PATENT APPLICATION

SYSTEM AND METHOD FOR VALIDATING SAFETY AND CORRECTNESS PROPERTIES OF USER-SUBMITTED PURE FUNCTIONS EXECUTING INSIDE A DATABASE THROUGH SANDBOXED CONTENT-ADDRESSED MODULES

Authors: Tom Tang

Organization: Edge Vector Foundation

Date: April 30, 2026

Applicant

Edge Vector Foundation
14205 SE 36th Street, Suite 100, Bellevue, WA 98006

Inventors

- Tom Tang, Seattle, WA

Field of the Invention

This invention relates to validated execution of user-submitted code within a database system, and more particularly to a system and method for providing best-effort validation of safety, termination, type correctness, and tamper detection for transforms executing inside a database through content-addressed identity, sandboxed execution, gas metering, deterministic semantics, and registration-time verification. The system cannot guarantee that code is bug-free or logically correct for a user's intended purpose, but provides strong mechanical guarantees around the properties it does verify.

Cross-Reference to Related Applications

None (provisional filing). *Note:* The security classification downgrade verification described herein (using Normalized Mutual Information analysis within the sandboxed execution environment) is related to, but distinct from, the information-theoretic transform classification described in the co-pending provisional application "System and Method for Automated Schema Deduplication

and Canonical Field Resolution Using Semantic Similarity with Information-Theoretic Transform Classification" by the same applicant. The present application claims the NMI analysis as enabled by and integrated with the sandboxed execution pipeline; the co-pending application claims it as part of a schema service classification system.

Background of the Invention

Databases have long supported computed fields and derived views. SQL expressions, stored procedures, and user-defined functions allow applications to define computations that run inside the database engine. However, these mechanisms suffer from fundamental limitations when user-submitted code is involved.

In traditional database systems, stored procedures and user-defined functions execute in the database's own runtime environment. They have access to the filesystem, network, other tables, and system resources. This makes user-submitted code a severe security risk: a malicious or buggy function can exfiltrate data, corrupt other records, exhaust system resources through infinite loops, or open network connections to external servers. As a result, most production databases either prohibit user-submitted code entirely or restrict it to a narrow set of built-in operations.

Systems that do allow user-submitted code typically execute it in a separate process with inter-process communication (IPC) overhead. This introduces latency, memory duplication, and serialization costs that make per-field transforms impractical at scale.

Recent systems have explored WebAssembly as an execution substrate for user-submitted code. Cloud edge platforms execute pure-function modules in sandboxed runtimes for serverless functions. Some database extensions enable sandboxed-bytecode user-defined functions. However, these systems do not provide the combination of: (1) content-addressed identity with tamper detection at every execution, (2) registration-time type verification before any data flows, (3) registration-time reversibility verification via round-trip testing, (4) parameterized domain-specific transforms with typed inputs and outputs, or (5) information-theoretic classification downgrade verification using the sandbox to safely analyze transform behavior on synthetic data.

Execution sandboxing itself — executing WebAssembly modules with restricted system access — is a known technique used in cloud edge computing, serverless platforms, and some database extensions. The present invention does not claim execution sandboxing in isolation. Rather, the invention claims the specific combination of: mandatory source code submission (no pre-compiled bytecode), AI-assisted source review before compilation, service-controlled deterministic compilation, content-addressed identity with per-execution tamper detection, and a multi-stage registration-time verification pipeline including type checking, reversibility testing, and information-theoretic classification analysis — all integrated as prerequisites before user-submitted code can execute inside a database.

Beyond security, existing systems provide no mechanism to verify that the code a user approved is the code that actually executes. A stored procedure can be silently modified after approval. There is no tamper detection, no immutability guarantee, and no way for third parties to independently verify what computation was performed.

Additionally, existing systems perform no registration-time verification of type compatibility.

A stored procedure that expects an integer but receives a string will fail at runtime, potentially after processing millions of records. There is no mechanism to verify before any data flows that a transform's input and output types are compatible with the fields it connects.

There is a need for a system that enables user-submitted code to execute inside a database with provable safety guarantees: verified identity, sandboxed execution, guaranteed termination, deterministic results, and type-checked interfaces — all verified before any data flows through the transform.

Summary of the Invention

The present invention provides a system and method for best-effort validation of user-submitted code for other users to run inside their database. The system provides strong guarantees around safety (sandboxing, termination, tamper detection) and type correctness, but cannot confirm that code is entirely bug-free or that its logic is correct for the user's intended purpose. The system comprises:

1. **Source code submission with service-side compilation** — the global transform registry accepts source code (not pre-compiled bytecode), compiles it using a pinned deterministic toolchain, and stores both source and compiled bytecode. This ensures the reviewable artifact (source) is the authoritative artifact that produces the executable.
2. **AI-assisted source code review** — before compilation, a large language model analyzes the submitted source for dangerous patterns, resource abuse, data exfiltration attempts, logic correctness, and code quality. Rejected submissions are returned with feedback; flagged submissions are queued for human review.
3. **Content-addressed transform identity** — each compiled transform is identified by the SHA-256 hash of its compiled bytecode, providing immutability, tamper detection, deduplication, and public auditability.
4. **Sandboxed execution with zero system access** — pure-function modules execute in a runtime with no filesystem, network, clock, or system access, and no access to database records or host memory outside the module's own isolated memory space.
5. **Gas metering for termination guarantee** — each module is assigned a fuel budget; every executed instruction consumes fuel; when fuel is exhausted, execution aborts, guaranteeing termination without infinite loops or resource exhaustion.
6. **Deterministic execution** — Execution semantics, including NaN canonicalization, produces identical output for identical input on every platform, enabling reproducible and independently verifiable transform results.
7. **Registration-time type verification** — before any data flows through a transform, the system verifies input/output type compatibility, parameter declaration validity, gas limit bounds, and optional invertibility via round-trip testing.
8. **Host-guest data passing protocol** — a defined protocol for passing typed data between the database host and the guest module through serialization, memory allocation within

the module's isolated memory, and deserialization, ensuring the module cannot access host memory.

9. **Parameterized transforms** — transforms accept optional configuration parameters (e.g., time windows, thresholds, aggregation granularity) declared at registration time and passed alongside input data at execution time, enabling a single transform module to serve multiple use cases.

10. **Security classification downgrade verification** — the sandboxed execution environment enables the system to safely run transforms on synthetic data at registration time and measure Normalized Mutual Information between input-output field pairs, verifying that claimed sensitivity downgrades reflect genuine information reduction rather than mere renaming.

Detailed Description of the Invention

Embodiments and Technology Independence

The invention is technology-agnostic with respect to the choice of compiled-bytecode format and sandboxed execution substrate. The properties recited in the claims — deterministic execution, isolated memory, content-addressable bytecode, gas-metered termination, and absence of host-system access — are realized by multiple existing and prospective execution technologies. **In one embodiment**, a registered transform module is compiled to WebAssembly and executed by a WebAssembly runtime configured with no system-call interface (no filesystem, network, or clock capabilities), NaN canonicalization, and fuel metering. WebAssembly is one suitable embodiment because its specification provides deterministic semantics across conforming runtimes, well-defined memory isolation, and an established toolchain for deterministic compilation from multiple source languages. In other embodiments, the registered transform module is compiled to a different bytecode format (for example, a deterministically configured Java Virtual Machine bytecode profile, eBPF, RISC-V machine code executed within a hardware-isolated context, or a content-addressable intermediate representation produced by a domain-specific compiler) and executed by a corresponding runtime that provides equivalent sandboxing, determinism, and metering guarantees. The claims are not limited to any single bytecode format or runtime implementation.

System Architecture

The system comprises four principal components: (1) a database engine that stores and queries data, (2) a pure-function transform execution engine embedded within the database, (3) a global transform registry service that accepts source code, compiles it, reviews it, and stores verified pure-function modules, and (4) an AI-assisted source code review system that analyzes submitted code for dangerous patterns before compilation.

Transform Model

Transforms are self-contained pure-function modules that perform complete, domain-specific computations. Each transform accepts a typed input (from one or more source fields), applies a compu-

tation, and produces a typed output. Transforms are not composable building blocks like arithmetic primitives — they are complete operations such as:

- "Average blood sugar over the last N days" (parameterized by time window)
- "Convert medical diagnosis to risk category" (classification)
- "Compute monthly spending by category" (aggregation)
- "Anonymize name to pseudonym hash" (irreversible redaction)

Transforms may accept optional parameters (e.g., time window, threshold, aggregation granularity) that configure their behavior. Parameters are declared at registration time and passed alongside the input data at execution time.

Source Code Submission and Service-Side Compilation

The global transform registry accepts *source code*, not pre-compiled bytecode. The service compiles the source to executable bytecode itself, ensuring that:

- **No hidden bytecode:** Developers cannot submit pre-compiled with obfuscated or malicious behavior that would not be apparent from source inspection. The service controls the compiler toolchain and compilation flags.
- **Source is the authority:** The source code is stored alongside the compiled bytecode. The service can recompile at any time to verify the compiled bytecode matches the source. Third parties can audit the source directly.
- **Deterministic compilation:** The service uses a pinned compiler version and configuration, ensuring that the same source always produces the same compiled bytecode on the service. This makes the content-addressed hash reproducible from source. Achieving deterministic compilation requires careful configuration: the compiler version, optimization flags, standard library version, and linker settings must all be pinned. The service maintains a locked toolchain configuration and verifies that recompilation of the same source produces the same bytecode hash. If a toolchain update would change output hashes, existing transforms are not affected — they retain their original hashes and bytecode. New submissions compiled under the updated toolchain receive new hashes.

The submission pipeline operates as follows:

1. Developer submits source code (e.g., Rust, AssemblyScript, or other source language with a deterministic compilation toolchain) along with declared input/output types and metadata.
2. The service performs AI-assisted source code review (described below).
3. If the review passes, the service compiles the source to executable bytecode using a pinned, deterministic compiler toolchain.

4. The service computes `SHA-256(wasm_bytes)` to derive the content-addressed identity.
5. The service runs the registration-time verification pipeline (type checking, parameter validation, gas validation, reversibility testing, classification analysis).
6. If all checks pass, the service stores both the source code and compiled bytecode, indexed by the content hash. The compiled bytecode is made available for nodes to fetch and execute.

AI-Assisted Source Code Review

Before compilation, the service submits the source code to a large language model for automated safety review. The AI agent analyzes the code for:

- **Dangerous patterns:** Attempts to exploit compiler bugs, unusual memory access patterns, obfuscated logic that could hide malicious behavior, or code that appears designed to circumvent the sandbox.
- **Resource abuse:** Algorithms with excessive computational complexity (e.g., exponential-time operations) that could consume disproportionate fuel even within gas limits, or memory allocation patterns designed to exhaust the isolated memory.
- **Data exfiltration attempts:** Code that encodes sensitive input data into output fields in non-obvious ways (e.g., steganographic encoding, timing-based side channels through computation patterns, or encoding data in floating-point precision bits).
- **Logic review:** Whether the code's behavior matches its declared description — for example, a transform described as "multiply by 0.85" that actually performs a different operation.
- **Code quality:** Potential bugs, undefined behavior, or edge cases that could produce incorrect results on certain inputs.

The AI review produces a structured report with a pass/flag/reject recommendation. Flagged submissions are queued for human review. Rejected submissions are returned to the developer with specific feedback. The AI review is a best-effort safety layer — it cannot guarantee the absence of all bugs or malicious behavior, but it catches common dangerous patterns that mechanical verification (type checking, gas limits) would miss.

Note: The AI review operates on source code, not the compiled bytecode. This is a critical design choice — source code is human-readable and amenable to semantic analysis, while compiled bytecode is opaque and much harder to reason about. By requiring source submission and performing review before compilation, the system ensures that the reviewable artifact (source) is the authoritative artifact that produces the executable bytecode.

Content-Addressed Transform Identity

After compilation, each transform is identified by `SHA-256(wasm_bytes)`. The hash IS the identity — there is no separate name or version number that could become inconsistent. Because the service controls compilation with a pinned toolchain, the same source always produces the same hash. This content-addressing provides:

- **Immutability:** A registered transform cannot be modified. Any change to the source produces different bytecode, which produces a different hash — a new transform identity.
- **Tamper detection:** Before every execution, the database node recomputes `SHA-256(wasm_bytes)` and compares it to the expected hash. If the bytes have been altered in storage or transit, the hash will not match and execution is rejected.
- **Deduplication:** Identical source code compiled by the same toolchain produces the same compiled bytecode and thus the same hash. The registry stores only one copy.
- **Full auditability:** Both source code and compiled bytecode are stored. Any party can read the source, recompile with the same toolchain, and verify the hash matches. The AI review report is also stored for audit.

Sandboxed Execution Model

The pure-function transform engine executes modules in a sandboxed runtime with the following constraints:

- **No filesystem access:** The module cannot read or write files on the host system. No filesystem access primitives are provided.
- **No network access:** The module cannot open sockets, make HTTP requests, or communicate with any external system. No network access primitives are provided.
- **No clock or time access:** The module cannot read the system clock, measure elapsed time, or access any time-related functions. This prevents timing-based side channels and ensures determinism.
- **No access to other database records:** The module receives only the serialized input bytes for the specific field being transformed. It cannot query other fields, other records, or other schemas.
- **No access to host memory:** The module operates within its own isolated memory. It cannot read or write any memory outside this sandbox. The host copies data into the module's isolated memory and reads results from it, but the module cannot access host memory.
- **Per-module memory limits:** Each module's isolated memory space is bounded to a configured maximum, preventing memory exhaustion attacks.

The module can only: (1) read the serialized input bytes that the host copied into its isolated memory space, (2) perform computation using its own code and isolated memory, and (3) write serialized output bytes into its isolated memory space for the host to read.

Gas Metering (Fuel-Based Termination Guarantee)

Each pure-function module is assigned a maximum fuel budget at registration time. The fuel limit is part of the transform's metadata and is verified to be within system-configured bounds during registration.

During execution, the sandboxed runtime consumes fuel for every instruction executed. When the fuel budget is exhausted, execution immediately aborts with a fuel-exhaustion error. This provides a hard termination guarantee:

- **No infinite loops:** Any loop that does not terminate within the fuel budget is aborted.
- **No resource exhaustion:** CPU consumption is bounded by the fuel budget regardless of the module's behavior.
- **Predictable execution cost:** The fuel budget provides an upper bound on execution time, enabling the database to schedule and prioritize transform execution.
- **Declared at registration:** The fuel limit is declared when the transform is registered, not at execution time. Users approving a transform can see its maximum execution cost before any data flows through it.

Deterministic Execution

The sandboxed runtime is configured for deterministic execution:

- **NaN canonicalization:** All NaN floating-point values are canonicalized to a single representation, eliminating platform-dependent NaN bit patterns.
- **No system-dependent behavior:** Without access to the clock, filesystem, network, or any system state, the module's output depends solely on its input bytes and its own code.
- **Platform independence:** the module specification guarantees identical semantics across all conforming runtimes. Combined with NaN canonicalization, this means the same module with the same input produces the same output on any platform.
- **Verifiable results:** Because execution is deterministic, any party with the pure-function module and the input can independently reproduce and verify the output.

Registration-Time Verification Pipeline

Before any data flows through a transform, the system performs a multi-step verification pipeline at registration time:

1. **Source code submission:** Developer submits source code with declared input/output types and metadata.
2. **AI-assisted source review:** The service submits the source to a large language model that analyzes it for dangerous patterns, resource abuse, data exfiltration attempts, logic correctness, and code quality. Rejected submissions are returned with feedback; flagged submissions are queued for human review.
3. **Compilation:** The service compiles the source to executable bytecode using a pinned, deterministic compiler toolchain.

4. **Hash computation:** Compute `SHA-256(wasm_bytes)` to derive the transform's identity.
5. **Deduplication check:** If a transform with the same hash is already registered, return the existing registration (identical code needs no re-verification).
6. **Type compatibility verification:** Verify that the transform's declared input type matches the source field's type and the declared output type matches the target field's type. Type mismatches are rejected before any data is processed.
7. **Parameter validation:** If the transform declares configurable parameters, verify that declared parameter types and default values are well-formed.
8. **Gas limit validation:** Verify that the declared maximum fuel is within system-configured bounds.
9. **Reversibility verification (optional):** If the transform declares itself reversible by exporting an inverse function, the system runs round-trip testing: generate type-appropriate synthetic inputs, run the forward transform, run the inverse transform on the forward output, and verify equality with the original input (a configurable floating-point tolerance (default 1×10^{-10})). If the round-trip fails, the transform is rejected as non-reversible.
10. **Classification downgrade verification (optional):** If the transform's output is intended to have a lower security classification than its inputs, the system verifies the downgrade by generating synthetic data, executing the transform in the sandbox, and measuring Normalized Mutual Information (NMI) between each input-output field pair. Output fields with NMI above a leakage threshold inherit the input's classification; output fields below the threshold are confirmed as genuinely reduced.

Only transforms that pass all verification steps are registered. The hash becomes the immutable identity, and the verified metadata (types, gas limit, reversibility status, declared parameters) is stored alongside the compiled bytecode.

Host-Guest Data Passing Protocol

Data is passed between the database (host) and the pure-function module (guest) through a defined protocol that ensures the module never accesses host memory:

1. **Serialization:** The host serializes the input `FieldValue` to bytes using a deterministic binary serialization format.
2. **Allocation:** The host calls the module's exported `alloc(size) -> ptr` function, which allocates space within the module's isolated memory and returns a pointer.
3. **Copy-in:** The host copies the serialized bytes into the module's isolated memory at the returned pointer.
4. **Invocation:** The host calls the module's exported `transform(ptr, len) -> output_ptr` function, passing the pointer and length of the input bytes.

5. **Guest computation:** The module deserializes the input, performs its computation, serializes the output, and returns a pointer to the output bytes in its isolated memory space.
6. **Copy-out:** The host reads the output bytes from the module's isolated memory.
7. **Deserialization:** The host deserializes the output bytes back to a `FieldValue`.

Each pure-function module exports the following functions:

- `alloc(size: u32) -> u32` — allocates `size` bytes in the module's isolated memory, returns pointer
- `transform(input_ptr: u32, input_len: u32) -> u32` — executes the forward transform
- `inverse(input_ptr: u32, input_len: u32) -> u32` (optional) — executes the inverse transform for reversible modules

The output pointer returned by `transform` or `inverse` encodes both the pointer and length of the output using a convention (e.g., the first 4 bytes at the pointer contain the length, followed by the output bytes).

Reversibility Verification

Transforms may optionally declare reversibility by exporting an `inverse` function in addition to the `transform` function. Reversibility is verified at registration time, not asserted by the developer:

1. The system generates type-appropriate synthetic test inputs (e.g., random floats for Float fields, random strings for String fields, random arrays for Array fields).
2. For each test input, the system runs the forward transform to produce an output.
3. The system runs the inverse transform on the forward output to produce a reconstructed input.
4. The system compares the reconstructed input to the original. For floating-point values, equality is checked with a configurable floating-point tolerance (default 1×10^{-10}). For all other types, exact equality is required.
5. If any test input fails the round-trip, the transform is rejected as non-reversible.

Only transforms that pass round-trip testing are marked as reversible in the registry. This verified reversibility enables write-back through the transform: when a user writes a value to a transform field, the database runs the inverse transform to compute the corresponding source value and writes it to the source field.

Security Classification Downgrade Verification

The sandboxed execution environment described above enables a novel verification capability: because transforms execute deterministically with gas-metered termination in a zero-access sandbox, the system can safely run the transform on synthetic data at registration time to measure its information-theoretic properties.

When a transform claims to reduce data sensitivity — for example, converting a detailed medical diagnosis (HIGH classification) into a boolean "has_chronic_condition" flag (LOW classification) — the system must verify that the claimed downgrade is genuine. A transform that simply renames "diagnosis" to "condition" preserves all information and should remain HIGH, while one that genuinely reduces specificity should be confirmed as LOW.

The system verifies classification downgrades at registration time using Normalized Mutual Information (NMI) analysis, leveraging the sandboxed execution environment to run the transform on synthetic data:

Phase 1 — Input Ceiling. The system resolves the data classification of each input field from the field registry. The maximum classification across all input fields establishes an upper bound (the *input ceiling*). A transform reading HIGH-classified fields cannot produce output classified higher than HIGH.

Phase 2 — NMI-Based Leakage Detection. The system generates N synthetic samples (512–1024) per input field, drawn from type-appropriate distributions:

- String fields: random alphanumeric strings or sentences from a fixed vocabulary
- Integer fields: uniform distribution over a plausible range
- Float fields: Gaussian with plausible mean and standard deviation
- Boolean fields: Bernoulli(0.5)
- Array fields: length 1–5, each element from the scalar distribution

For each sensitive input field X_i, the system varies X_i across N samples while holding all other fields at fixed baseline values. The transform module executes on each sample (inside the same sandbox with gas metering). For each input-output field pair (X_i, Y_j), the system computes Normalized Mutual Information:

$$\mathrm{NMI}(X_i, Y_j) = \frac{I(X_i; Y_j)}{\sqrt{H(X_i) \cdot H(Y_j)}}$$

where $I(X_i; Y_j)$ is the mutual information estimated from discretized histograms and H is Shannon entropy. NMI ranges from 0 (statistically independent) to 1 (deterministic function of the input).

Classification Assignment. For each output field Y_j, input fields with NMI $> \theta$ (default $\theta = 0.1$) are identified as *leaking* into that output. The output field inherits the maximum classification of all leaking inputs. The overall transform classification is the maximum across all output fields.

This approach is grounded in the *Data Processing Inequality*: for any Markov chain $X \to$ transform $\to Y$, $I(X;Y) \leq H(X)$. A transform can only discard information, never create it. Measuring actual NMI reveals the true leakage.

The key property is that NMI is a *black-box* measure — it detects information preservation regardless of the transform's internal implementation. A transform that renames a field, encodes it differently, or partially preserves it through bucketing will show elevated NMI, correctly identifying the leakage without requiring static analysis of the compiled bytecode.

Deterministic Execution Guarantees

The combination of module semantics, NaN canonicalization, and the absence of any system access produces fully deterministic execution. For any registered transform identified by hash h and any input bytes b:

$$\texttt{transform}_h(b) = c \quad \forall \text{ platforms}, \forall \text{ time}$$

This determinism enables:

- **Reproducibility:** Any node with the same pure-function module and input produces the same output. Results can be independently verified.
- **Caching:** Results can be cached by (h, b) without invalidation concerns from nondeterminism.
- **Distributed verification:** Multiple nodes can independently compute and compare results to detect tampering.

Claims

1. A computer-implemented method for validating safety and correctness properties of user-submitted code executing inside a database, comprising:
    a. receiving source code for a transform from a developer, along with declared input/output types;
    b. performing AI-assisted source code review by submitting the source to a large language model that analyzes it for dangerous patterns, resource abuse, data exfiltration attempts, and logic correctness;
    c. compiling the source code to executable bytecode using a pinned, deterministic compiler toolchain controlled by the service;
    d. computing a SHA-256 hash of the compiled module's bytecode to derive a content-addressed identity for the transform;

e. performing registration-time verification comprising type compatibility checking, parameter declaration validation, and gas limit validation, all before any data flows through the transform;

f. storing both the source code and compiled module in a registry indexed by the content-addressed hash.

2. The method of claim 1, further comprising at execution time:

   a. recomputing the SHA-256 hash of the stored module and comparing to the expected hash, rejecting execution if the hashes do not match;

   b. executing the module in a sandboxed runtime with no filesystem, network, clock, or system access, and no access to database records or host memory outside the module's own isolated memory space.

3. The method of claim 1, wherein the source code is reviewed before compilation by a large language model as part of a source-only submission pipeline (no pre-compiled bytecode is accepted), comprising:

   a. submitting the source code to a large language model before compilation;

   b. analyzing the code for dangerous patterns including attempts to exploit compiler bugs, obfuscated logic, and sandbox circumvention techniques;

   c. analyzing for resource abuse including algorithms with excessive computational complexity and memory allocation patterns designed to exhaust the isolated memory;

   d. analyzing for data exfiltration attempts including steganographic encoding of input data in output fields and encoding data in floating-point precision bits;

   e. verifying that the code's behavior matches its declared description;

   f. producing a structured report with a pass, flag, or reject recommendation, where flagged submissions are queued for human review and rejected submissions are returned to the developer with specific feedback.

4. The method of claim 1, wherein the sandboxed runtime enforces the following constraints on the executing module:

   a. no filesystem access primitives are provided, preventing file read or write operations;

   b. no network access primitives are provided, preventing socket or HTTP operations;

   c. no clock or time functions are provided, preventing timing-based side channels;

   d. the module operates exclusively within its own isolated memory space, which is bounded to a configured maximum size;

   e. the module receives only serialized input bytes for the specific field being transformed and cannot access any other database records, fields, or schemas.

5. The method of claim 1, further comprising gas metering for termination guarantee, wherein:

   a. each module is assigned a maximum fuel budget declared at registration time;

   b. the fuel budget is verified to be within system-configured bounds during registration;

   c. during execution, every executed instruction consumes fuel from the budget;

d. when the fuel budget is exhausted, execution immediately aborts with a fuel-exhaustion error;

e. the fuel budget provides an upper bound on execution time regardless of the module's behavior.

6. The method of claim 1, wherein execution is deterministic such that for any registered transform identified by hash h and any input bytes b, the transform produces identical output on every platform and at every time, achieved through:

   a. NaN canonicalization of all floating-point values to a single representation;

   b. absence of system-dependent state access (no clock, filesystem, network, or system calls);

   c. module specification guarantees of identical semantics across all conforming runtimes.

7. The method of claim 1, wherein registration-time type verification comprises:

   a. verifying that the transform's declared input type matches the source field's data type;

   b. verifying that the transform's declared output type matches the target field's data type;

   c. rejecting transforms with type mismatches before any data is processed through the transform.

8. The method of claim 1, further comprising reversibility verification, wherein:

   a. a transform optionally exports an inverse function in addition to the forward transform function;

   b. at registration time, the system generates type-appropriate synthetic test inputs;

   c. for each test input, the system runs the forward transform followed by the inverse transform;

   d. the system verifies that the round-trip reconstructed input matches the original input, with a configurable floating-point tolerance (default 1×10^{-10});

   e. transforms that fail round-trip testing are rejected as non-reversible;

   f. verified-reversible transforms enable write-back through the inverse transform to mutate source data.

9. The method of claim 1, wherein data passing between the database host and the pure-function module occurs through a defined protocol comprising:

   a. the host serializes a typed field value to bytes using a deterministic binary format;

   b. the host calls the module's exported `alloc(size)` function to allocate space within the module's isolated memory;

   c. the host copies the serialized bytes into the module's isolated memory at the allocated pointer;

   d. the host calls the module's exported `transform(ptr, len)` function;

   e. the module deserializes the input, computes the transform, serializes the output within its isolated memory space;

   f. the host reads the output bytes from the module's isolated memory and deserializes them to a typed field value;

g. at no point does the module access host memory outside its own isolated memory.

10. The method of claim 1, further comprising security classification downgrade verification, wherein:

    a. each input field carries a data sensitivity classification from a field registry;
    b. the transform module executes within the sandboxed runtime described in claim 2 with gas metering during the synthetic data analysis, ensuring safe and bounded execution of untrusted code on test data;
    c. an input ceiling is established as the maximum classification across all input fields;
    d. the system generates N type-appropriate synthetic samples per input field;
    e. for each sensitive input field X_i, the system varies X_i across the N samples while holding other fields at fixed baseline values, executing the transform module in the sandbox on each sample;
    f. for each input-output field pair (X_i, Y_j), the system computes Normalized Mutual Information: $\mathrm{NMI}(X_i, Y_j) = I(X_i; Y_j)/\sqrt{H(X_i) \cdot H(Y_j)}$;
    g. output fields with NMI above a leakage threshold (default $\theta = 0.1$) inherit the maximum classification of all leaking input fields;
    h. output fields with NMI below the threshold for all input fields are confirmed as genuinely reduced in sensitivity;
    i. the overall transform classification is the maximum across all output field classifications;

    such that the system verifies claimed sensitivity downgrades using a black-box information-theoretic measure that detects information preservation regardless of the transform's internal implementation.

11. A system for validating safety and correctness properties of user-submitted code executing inside a database, comprising:

    a. a database engine configured to store and query data;
    b. a global transform registry service configured to accept source code submissions, perform AI-assisted source code review, compile source to executable bytecode using a pinned deterministic toolchain, and store both source code and compiled pure-function modules indexed by the SHA-256 hash of their bytecode, with associated verified metadata including input/output types, declared parameters, gas limits, reversibility status, and AI review reports;
    c. a registration-time verification pipeline configured to verify type compatibility, validate declared parameters, validate gas limits, and optionally verify reversibility via round-trip testing, all before any data flows through a transform;
    d. a sandboxed runtime configured to execute transforms with no filesystem, network, clock, or system access, with gas metering for termination guarantee, NaN canonicalization for deterministic execution, and per-module memory limits;
    e. a host-guest data passing subsystem configured to serialize typed field values, allocate space within the pure-function module's isolated memory space, copy data in, invoke the transform, and read results back, without the module accessing host memory;

f. a content verification subsystem configured to recompute the SHA-256 hash of a module's bytecode before every execution and reject execution if the hash does not match the expected identity;

g. a classification downgrade verification subsystem configured to execute transforms on synthetic data within the sandbox, measure Normalized Mutual Information between input-output field pairs, and verify that claimed sensitivity downgrades reflect genuine information reduction.

Abstract

A system and method for best-effort validation of user-submitted code for execution inside a database. The global transform registry accepts source code, performs AI-assisted review using a large language model to detect dangerous patterns, compiles to executable bytecode using a pinned deterministic toolchain, and runs a multi-stage verification pipeline before making the transform available. The system provides strong mechanical guarantees around safety, termination, type correctness, and tamper detection, but cannot confirm that code is entirely bug-free or logically correct for a user's intended purpose. Each transform is identified by the SHA-256 hash of its compiled bytecode, providing immutability, tamper detection, deduplication, and public auditability. Transforms execute in a sandboxed runtime with no filesystem, network, clock, or system access, and no access to database records or host memory outside the module's own isolated memory space. Gas metering assigns each module a fuel budget that limits instruction execution, guaranteeing termination without infinite loops. NaN canonicalization and the absence of system access produce fully deterministic execution — identical input always yields identical output on every platform. A registration-time verification pipeline checks type compatibility, parameter declaration validity, gas limits, optional reversibility via round-trip testing, and optional security classification downgrade verification before any data flows through a transform. Transforms are self-contained pure-function modules that perform complete, domain-specific computations with typed inputs and typed outputs, and may accept optional configuration parameters (e.g., time windows, thresholds, aggregation granularity) declared at registration time. Classification downgrades are verified by executing the transform on synthetic data within the sandbox and measuring Normalized Mutual Information between each input-output field pair — a black-box information-theoretic measure that detects information preservation regardless of the transform's internal implementation. Data passes between the database host and the guest module through a defined protocol using serialization, isolated memory allocation, and deserialization.

Prepared for provisional patent filing by Edge Vector Foundation.
Filing date: [TO BE DETERMINED]
This document constitutes a provisional patent application under 35 U.S.C. 111(b).

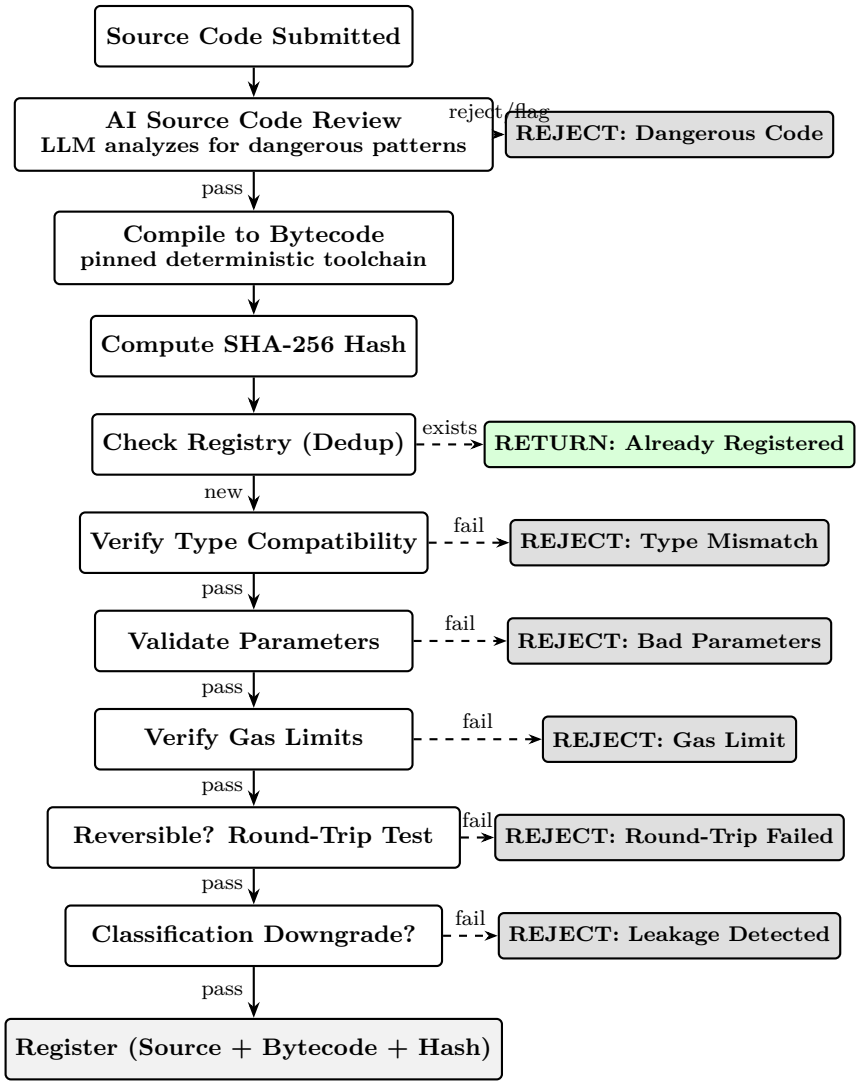

Figure 1: **Registration-Time Verification Pipeline.** The compiled bytecode passes through a multi-step verification pipeline before registration. Each step can reject the transform. Only transforms that pass all checks are registered with their SHA-256 hash as identity.

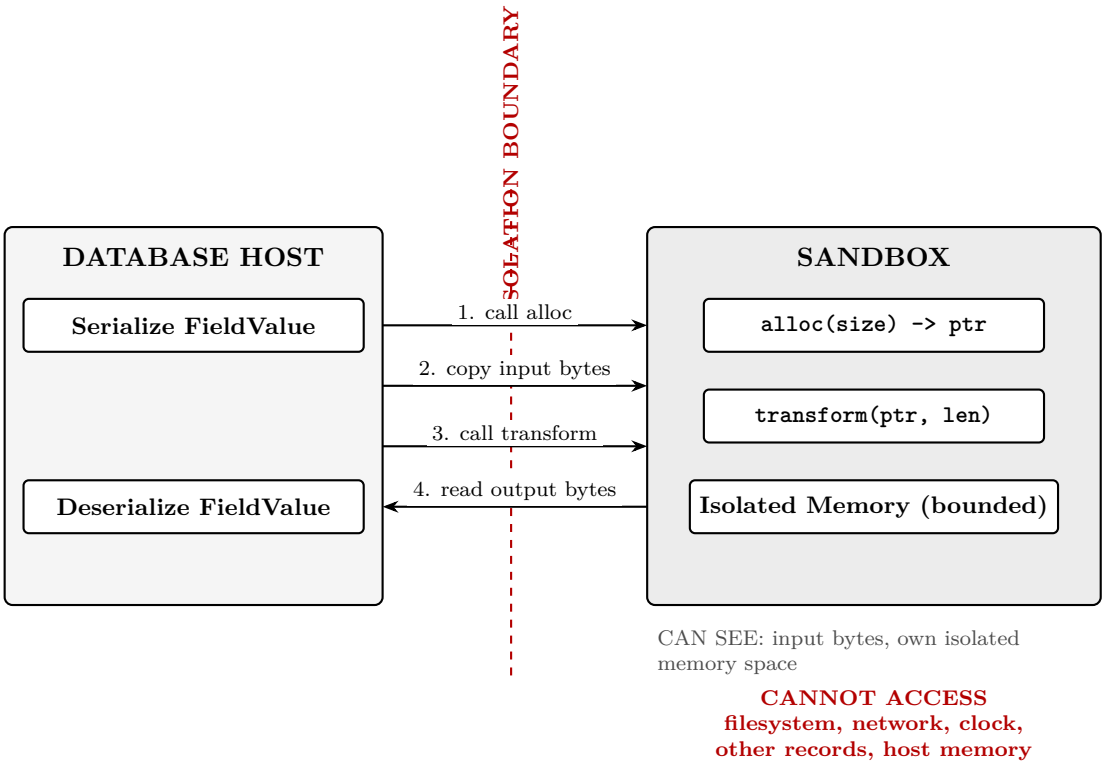

Figure 2: **Sandboxed Execution Model.** The database host serializes input, copies it into the execution sandbox via `alloc()`, calls `transform()`, and reads the output. The sandbox can only see its input bytes and its own isolated memory space. It cannot access the filesystem, network, clock, other database records, or host memory.

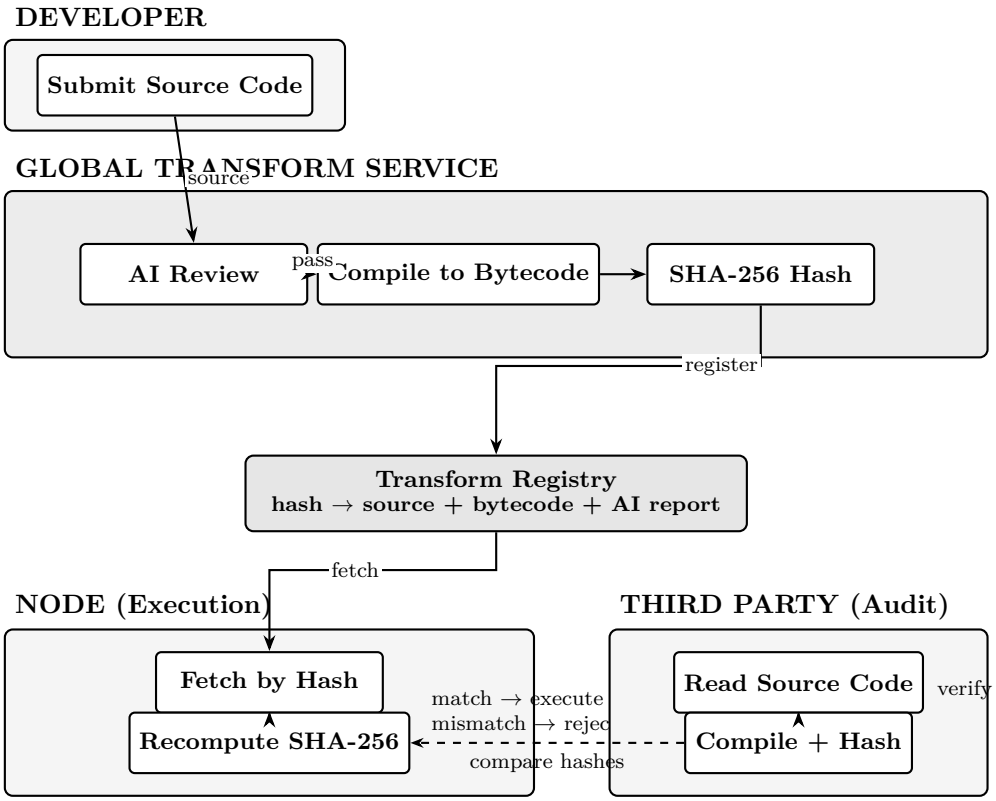

Figure 3: **Source-to-Execution Pipeline.** A developer submits source code. The global transform service performs AI review, compiles to executable bytecode, and registers by hash. Nodes fetch the compiled bytecode by hash and recompute the hash before execution. Third parties can read the source, recompile, and verify the hash matches.

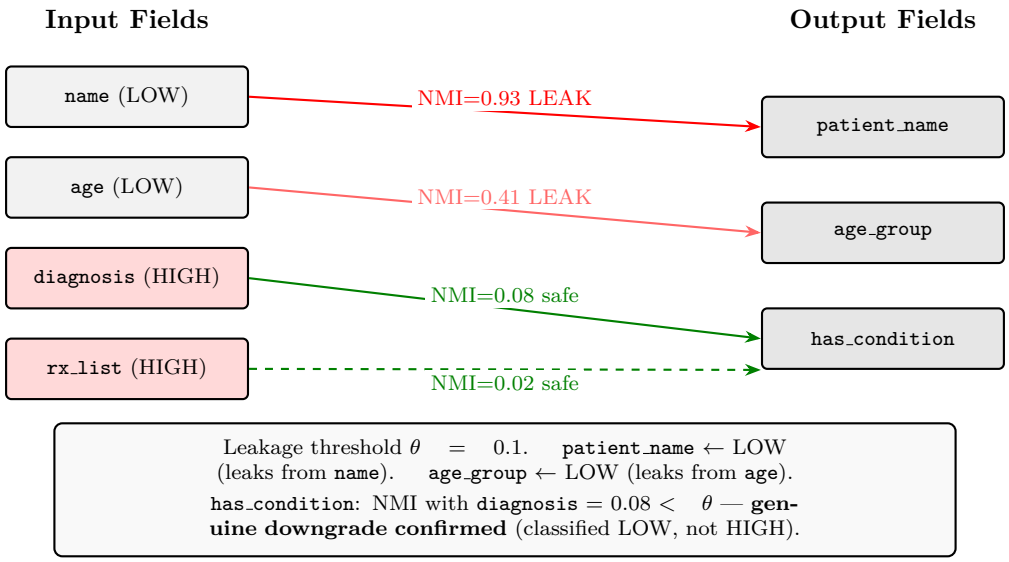

Figure 4: **Classification Downgrade Verification.** The system runs the transform module on synthetic data inside the sandbox and measures Normalized Mutual Information between each input-output field pair. Red arrows indicate information leakage (NMI $> \theta$); green arrows confirm genuine information reduction (NMI $< \theta$). Output fields inherit the maximum classification of leaking inputs.